

Búsqueda no informada II

DFS, profundización iterativa y costo uniforme

Joaquín F. Sánchez

Curso de Inteligencia Artificial

2026

Al finalizar la sesión, el estudiante estará en capacidad de:

- explicar el funcionamiento de DFS, profundización iterativa y costo uniforme;
- relacionar cada estrategia con la estructura usada para organizar la frontera;
- ejecutar manualmente los algoritmos sobre árboles y grafos pequeños;
- implementar versiones básicas en Python;
- comparar completitud, optimalidad y complejidad;
- seleccionar la estrategia apropiada según el problema.

Búsqueda en anchura (BFS)

Expande primero el nodo menos profundo mediante una cola FIFO. Es completa y, con costos unitarios, encuentra una solución con el menor número de pasos.

Pregunta orientadora

¿Qué cambia si priorizamos profundidad, aumentamos gradualmente un límite o elegimos el camino acumulado más barato?

La frontera determina la estrategia

Estrategia	Criterio de selección	Estructura
BFS	menor profundidad	cola FIFO
DFS	mayor profundidad	pila LIFO
Iterativa	DFS con límite creciente	pila/recursión
Costo uniforme	menor costo acumulado $g(n)$	cola de prioridad

Idea clave

Los algoritmos comparten el ciclo de explorar, probar el objetivo y expandir; cambia el orden en que se seleccionan los nodos.

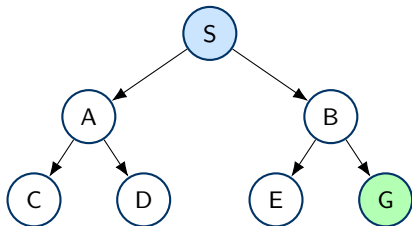
Búsqueda en profundidad (DFS)

Definición

DFS expande primero uno de los nodos más profundos de la frontera. Continúa por una rama hasta alcanzar una meta o hasta que no pueda avanzar; entonces retrocede.

- usa una pila LIFO o llamadas recursivas;
- requiere poca memoria en comparación con BFS;
- el orden de los sucesores afecta fuertemente el recorrido;
- necesita control de ciclos cuando se aplica a grafos.

DFS: exploración de una rama



Si se recorren sucesores de izquierda a derecha:

S, A, C, D, B, E, G

Observación

DFS puede llegar tarde a una meta poco profunda si explora primero otra rama extensa.

La pila LIFO en DFS

Para visitar primero el sucesor izquierdo, se insertan los sucesores en orden inverso.

Paso	Nodo extraído	Pila: cima a la izquierda
0	–	[<i>S</i>]
1	<i>S</i>	[<i>A</i> , <i>B</i>]
2	<i>A</i>	[<i>C</i> , <i>D</i> , <i>B</i>]
3	<i>C</i>	[<i>D</i> , <i>B</i>]
4	<i>D</i>	[<i>B</i>]
5	<i>B</i>	[<i>E</i> , <i>G</i>]
6	<i>E</i>	[<i>G</i>]
7	<i>G</i>	objetivo encontrado

Pseudocódigo de DFS sobre grafos

```
DFS(problema):  
  frontera <- pila([nodo_inicial])  
  visitados <- conjunto_vacio  
  
  mientras frontera no este vacia:  
    actual <- frontera.desapilar()  
    si objetivo(actual.estado): retornar actual  
  
    si actual.estado no esta en visitados:  
      visitados.agregar(actual.estado)  
      para cada hijo en sucesores_en_orden_inverso(actual):  
        si hijo.estado no esta en visitados:  
          frontera.apilar(hijo)  
  
  retornar fallo
```


Implementación iterativa de DFS en Python

```
def dfs(grafo, inicio, objetivo):
    pila = [(inicio, [inicio])]
    visitados = set()

    while pila:
        actual, camino = pila.pop()
        if actual == objetivo:
            return camino
        if actual in visitados:
            continue
        visitados.add(actual)

        for vecino in reversed(grafo.get(actual, [])):
            if vecino not in visitados:
                pila.append((vecino, camino + [vecino]))
    return None
```

Propiedades de DFS

Sea b el factor de ramificación y m la profundidad máxima.

Criterio	Resultado
Compleitud	No, en espacios infinitos o con ciclos sin control
Optimalidad	No
Tiempo	$O(b^m)$
Espacio	$O(bm)$

Fortaleza principal

Su consumo de memoria crece aproximadamente con la profundidad, no con todo el ancho de la frontera.

Riesgos y controles en DFS

- **Ciclos:** usar visitados o comprobar estados en el camino actual.
- **Ramas infinitas:** establecer un límite de profundidad.
- **Soluciones no óptimas:** no detenerse en la primera si se requiere el menor costo.
- **Dependencia del orden:** definir explícitamente el orden de sucesores.

Precaución

Un conjunto global de visitados ahorra trabajo, pero en ciertas variantes con límites puede impedir reconsiderar un estado alcanzado posteriormente con menor profundidad.

Búsqueda con profundidad limitada

Idea

Es una variante de DFS que no expande nodos más allá de un límite ℓ .

Sus posibles resultados son:

Solución: se encontró una meta dentro del límite.

Corte: podría existir una solución por debajo del límite.

Fallo: no existe solución en el espacio explorado.

Utilidad

Evita descender indefinidamente, pero exige escoger un límite adecuado.

Profundización iterativa (IDDFS)

Definición

Ejecuta repetidamente búsqueda en profundidad limitada con límites $0, 1, 2, \dots$ hasta encontrar una solución.

$$\text{DLS}(0) \rightarrow \text{DLS}(1) \rightarrow \text{DLS}(2) \rightarrow \dots$$

- encuentra primero la meta de menor profundidad;
- conserva el bajo consumo de memoria de DFS;
- no requiere conocer previamente la profundidad de la solución.

Para el árbol anterior, con meta G a profundidad 2:

Límite	Recorrido	Resultado
0	S	corte
1	S, A, B	corte
2	S, A, C, D, B, E, G	solución

Pregunta frecuente

¿Repetir niveles es muy costoso? Usualmente no: en árboles con $b > 1$, la mayoría de los nodos se encuentra en el nivel más profundo.

Pseudocódigo de profundización iterativa

```
IDDFS(problema):  
  para limite <- 0, 1, 2, ...:  
    resultado <- DLS(nodo_inicial, limite)  
    si resultado es solucion: retornar resultado  
    si resultado es fallo: retornar fallo
```

```
DLS(nodo, limite):  
  si objetivo(nodo.estado): retornar nodo  
  si limite = 0: retornar corte  
  para cada hijo en expandir(nodo):  
    resultado <- DLS(hijo, limite - 1)  
    si resultado es solucion: retornar resultado  
  retornar corte o fallo, segun los resultados
```

Implementación compacta de IDDFS

```
def dls(grafo, actual, objetivo, limite, camino):
    if actual == objetivo:
        return camino
    if limite == 0:
        return None
    for vecino in grafo.get(actual, []):
        if vecino not in camino: # evita ciclos en la ruta
            r = dls(grafo, vecino, objetivo,
                    limite - 1, camino + [vecino])
            if r is not None:
                return r
    return None

def iddfs(grafo, inicio, objetivo, maximo):
    for limite in range(maximo + 1):
        r = dls(grafo, inicio, objetivo, limite, [inicio])
        if r is not None:
            return r
    return None
```


Propiedades de IDDFS

Sea d la profundidad de la solución más cercana.

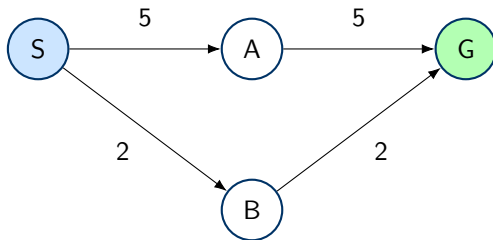
Criterio	Resultado
Complejidad	Sí, si b es finito
Optimalidad	Sí, cuando cada paso tiene igual costo
Tiempo	$O(b^d)$
Espacio	$O(bd)$

Balance

IDDFS combina la exploración por profundidad de BFS con el uso reducido de memoria de DFS.

Cuando el número de pasos no basta

Dos rutas pueden tener diferente número de acciones y diferente costo:



Ruta $S \rightarrow A \rightarrow G$: costo 10. Ruta $S \rightarrow B \rightarrow G$: costo 4.

Búsqueda de costo uniforme (UCS)

Definición

UCS expande el nodo de la frontera cuyo camino desde el inicio tiene el menor costo acumulado $g(n)$.

- usa una cola de prioridad mínima;
- generaliza BFS para costos de acción diferentes;
- es equivalente al algoritmo de Dijkstra en su uso habitual sobre grafos;
- no utiliza una estimación heurística del costo restante.

$$g(hijo) = g(padre) + c(padre, accion, hijo)$$

UCS: expansión por costo acumulado

Paso	Nodo extraído	Frontera (<i>costo, nodo</i>)
0	–	(0, S)
1	S	(2, B), (5, A)
2	B	(4, G), (5, A)
3	G	objetivo con costo 4

Regla crítica

La prueba de objetivo se realiza cuando el nodo se extrae de la cola de prioridad, no cuando se genera. Así se garantiza que no existe otro camino pendiente más barato.

Actualizar un estado con un camino más barato

Si un estado ya está en la frontera y aparece un camino con menor costo:

- 1 se actualiza su mejor costo conocido;
- 2 se registra el nuevo padre;
- 3 se agrega la nueva entrada a la cola de prioridad;
- 4 al extraer una entrada obsoleta, se descarta.

Estructuras habituales

Una cola de prioridad almacena (*costo*, *estado*) y un diccionario mantiene *mejor*[*estado*].

Pseudocódigo de costo uniforme

```
UCS(problema):  
  frontera <- prioridad((0, nodo_inicial))  
  mejor[inicial] <- 0  
  
  mientras frontera no este vacia:  
    (costo, actual) <- extraer_minimo(frontera)  
    si costo > mejor[actual.estado]: continuar  
    si objetivo(actual.estado): retornar actual  
  
    para cada (hijo, costo_accion) en expandir(actual):  
      nuevo <- costo + costo_accion  
      si hijo.estado no esta en mejor o nuevo < mejor[hijo.estado]:  
        mejor[hijo.estado] <- nuevo  
        padre[hijo.estado] <- actual.estado  
        insertar(frontera, (nuevo, hijo))  
  retornar fallo
```

Implementación base de UCS en Python

```
import heapq

def ucs(grafo, inicio, objetivo):
    frontera = [(0, inicio, [inicio])]
    mejor = {inicio: 0}

    while frontera:
        costo, actual, camino = heapq.heappop(frontera)
        if costo != mejor.get(actual):
            continue
        if actual == objetivo:
            return costo, camino
        for vecino, paso in grafo.get(actual, []):
            nuevo = costo + paso
            if nuevo < mejor.get(vecino, float("inf")):
                mejor[vecino] = nuevo
                heapq.heappush(frontera,
                              (nuevo, vecino, camino + [vecino]))

    return None
```

Propiedades de UCS

Sea C^* el costo óptimo y $\varepsilon > 0$ el costo mínimo de una acción.

Criterio	Resultado
Compleitud	Sí, si cada paso cuesta al menos ε
Optimalidad	Sí, con costos no negativos
Tiempo	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$
Espacio	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$

Condición esencial

Los costos negativos invalidan la garantía habitual de UCS.

Comparación de las estrategias

Algoritmo	Selección	Completo	Óptimo	Memoria
BFS	menor profundidad	sí	costos iguales	alta
DFS	mayor profundidad	no en general	no	baja
IDDFS	límite creciente	sí	costos iguales	baja
UCS	menor $g(n)$	sí*	sí*	alta

* Bajo las condiciones indicadas: costos no negativos y costo de paso mínimo positivo para completitud.

Selección rápida

Poca memoria y solución profunda: DFS. Profundidad desconocida y costos iguales: IDDFS. Costos diferentes y se requiere mínimo costo: UCS.

- confundir DFS con BFS por insertar sucesores en un orden incorrecto;
- omitir el control de ciclos en grafos;
- considerar óptima la primera solución de DFS;
- usar IDDFS sin distinguir corte de fallo;
- hacer la prueba de meta al generar un nodo en UCS;
- marcar como definitivo un costo antes de extraer su mejor entrada;
- aplicar UCS a aristas con costos negativos.

Considere un grafo ponderado con inicio S y objetivo G :

$$S \rightarrow A : 2, S \rightarrow B : 1, A \rightarrow C : 2, B \rightarrow D : 2, C \rightarrow G : 5, D \rightarrow G : 2$$

Use el orden alfabético para los empates.

- 1 Determine el orden de expansión con DFS.
- 2 Ejecute IDDFS hasta encontrar G .
- 3 Registre la cola de prioridad de UCS en cada paso.
- 4 Compare las rutas y sus costos.
- 5 Justifique cuál algoritmo elegiría si la memoria fuera limitada.

- ❶ ¿Por qué DFS consume menos memoria que BFS?
- ❷ ¿Cómo evita IDDFS depender de un límite elegido manualmente?
- ❸ ¿Por qué UCS no debe comprobar la meta al generar el nodo?
- ❹ ¿En qué caso BFS y UCS producen el mismo comportamiento?
- ❺ ¿Qué algoritmo usaría para minimizar saltos? ¿Y para minimizar costo?

- DFS prioriza profundidad mediante una pila y ahorra memoria.
- La búsqueda limitada controla ramas profundas o infinitas.
- IDDFS aumenta el límite y combina ventajas de BFS y DFS.
- UCS selecciona el camino acumulado más barato con una cola de prioridad.
- La completitud y optimalidad dependen de las condiciones del problema.
- Elegir una estrategia exige considerar costos, profundidad y memoria.

Próximo paso: comparar búsqueda no informada y búsqueda heurística